랭체인으로 RAG 시작하기

제작 : 박광석(모두의연구소, https://www.linkedin.com/in/andkspark)

해당 노트는 Langchain으로 RAG를 구현하기 위해 필요한 각 컴포넌트인 Document Loaders, Text splitters, Text embeddings, Vectorstores, Retriever를 다룹니다

```
# 코드로 형식 지정됨
```

Step 0 : 설치와 준비

Langchain 설치 및 Gemini API 키를 등록하도록 합니다.

```
OPENAI_KEY = "sk-"

import os
os.environ["OPENAI_API_KEY"] = OPENAI_KEY
```

```
! curl ipinfo.io
```

```
{
  "ip": "35.189.171.71",
  "hostname": "71.171.189.35.bc.googleusercontent.com",
  "city": "Taipei",
  "region": "Taiwan",
  "country": "TW",
  "loc": "25.0531,121.5264",
  "org": "AS396982 Google LLC",
  "timezone": "Asia/Taipei",
  "readme": "https://ipinfo.io/missingauth"
}
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", forc
```

```
YOUR_API_KEY = 'sk-'
```

```
!pip install U -q langchain langchain-google-genai
!pip install U -q langchain-community langchain-core
```

```
import os
os.environ['GOOGLE_API_KEY'] = YOUR_API_KEY
```

```
#랭체인의 구글Api를 사용합니다
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model = "gemini-1.5-flash", temperature=0.0)
```

```
# !pip install -q pypdf pdf2image docx2txt pdfminer unstructured #의존성 모듈을 설치합니다
```

Step 1 : Document Loaders 사용해보기

Document Loader는 다양한 형태의 원본 데이터를
LLM이 이해할 수 있는 Document 객체(text + metadata) 로 변환하는 역할을 합니다.

PDF, 웹페이지, CSV와 같이 형식이 서로 다른 문서들을 일관된 구조로 파싱하여, 이후 Chunking·Embedding·검색(Retrieval) 단계에서 바로 사용할 수 있도록 만들어줍니다.

즉, Document Loader는 **RAG 파이프라인의 가장 첫 단계에서 "데이터를 읽을 수 있는 형태로 정리하는 역할을 담당**합니다.

공식 문서에서는 지원되는 다양한 Loader 목록을 확인할 수 있습니다.
https://python.langchain.com/docs/modules/data_connection/document_loaders/

PDFLoader 사용

이번 실습에서는 가장 많이 사용되는 문서 형식인 PDF 파일을 대상으로 PyPDFLoader를 사용해 문서를 불러옵니다.

실습을 위해, 질의응답에 활용하고 싶은 PDF 파일을 먼저 Colab 환경(또는 Drive)에 업로드해주세요.

PDFLoader는 각 페이지를 하나의 Document 단위로 변환하며, 이 단계에서 생성된 문서들은 이후 Text Splitter를 통해 의미 단위로 다시 분할됩니다.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("/content/Demian-By-Hermann-Hesse.pdf")
pages = loader.load_and_split()
```

```
pages[0]
```

```
Document(metadata={'producer': 'Adobe Acrobat Standard DC 19 Paper Capture Plug-in', 'creator': 'ScanFix(TM)
Enhanced', 'creationdate': '2015-09-10T01:40:29+00:00', 'moddate': '2019-01-30T17:47:47+01:00', 'source':
'/content/Demian-By-Hermann-Hesse.pdf', 'total_pages': 182, 'page': 0, 'page_label': '1'}, page_content='DEMIAN
\n• \nDownloaded from https://www.holybooks.com')
```

```
print(pages[10])
```

```
page_content='TWO WOR.LDS
Finally, out of sheer nervousness, I began to talk. I
invented a long story of robbery, in which I featured as
the hero. One night in the comer by the mill a friend
and I ha.d stolen a whole sackful of apples, not just
ordinary apples but pippins, golden pippins of the best
kind at that. I was taking refuge in my story from the
dangers of the moment and found no difficulty in invent
ing and relating it. In order not to dry up too soon and
perhaps become involved in something worse, I gave full
rein to my narrative powers. One of us, I reported, had
always stood guard while the other sat in the tree and
chucked the apples down, and the sack had got so heavy
that in the end we had to open it and leave half behind,
but we came back half an hour later and fetched them
too.
I hoped for some applause at the end of my story; I
had warmed up to the narrative aJ: last, carried away by
my own eloquence. The two smaller boys were silent,
waiting, Lut Franz Kromer gave me a penetrating look
through his narrow,-1 eyes. "Is that yarn true?" he
asked in a menacing tone.
"Yes," I said.
"Really and truly?"
"Yes, really and truly," I asserted defiantly while I
choked inwardly with fear.
"Can you swear to it?"
I was very afraid but I said 'Yes' without hesitation.
"Hand on your heart?"
"Hand on my heart,"
"Right then," he said and turned away.
I thought this was all very satisfactory and I was glad
15
Downloaded from https://www.holybooks.com' metadata={'producer': 'Adobe Acrobat Standard DC 19 Paper Capture Plu
```

출력 결과를 보기 쉽게 확인하기 위해, Document 객체 전체가 아닌 실제 텍스트 본문이 담긴 page_content만 선택하여 확인해보겠습니다.

```
print(pages[10].page_content)
```

```
TWO WOR.LDS
Finally, out of sheer nervousness, I began to talk. I
invented a long story of robbery, in which I featured as
the hero. One night in the comer by the mill a friend
and I ha.d stolen a whole sackful of apples, not just
ordinary apples but pippins, golden pippins of the best
kind at that. I was taking refuge in my story from the
dangers of the moment and found no difficulty in invent
ing and relating it. In order not to dry up too soon and
perhaps become involved in something worse, I gave full
rein to my narrative powers. One of us, I reported, had
always stood guard while the other sat in the tree and
chucked the apples down, and the sack had got so heavy
that in the end we had to open it and leave half behind,
but we came back half an hour later and fetched them
too.
I hoped for some applause at the end of my story; I
had warmed up to the narrative aJ: last, carried away by
my own eloquence. The two smaller boys were silent,
waiting, Lut Franz Kromer gave me a penetrating look
through his narrow,-1 eyes. "Is that yarn true?" he
asked in a menacing tone.
"Yes," I said.
"Really and truly?"
```

```
"Yes, really and truly," I asserted defiantly while I
choked inwardly with fear.
"Can you swear to it?"
I was very afraid but I said 'Yes' without hesitation.
"Hand on your heart?"
"Hand on my heart,"
"Right then," he said and turned away.
I thought this was all very satisfactory and I was glad
15
Downloaded from https://www.holybooks.com
```

CSVLoader

SV 파일은 행(row) 단위로 구조화된 데이터를 담고 있는 형식으로, LangChain의 CSVLoader를 사용하면 각 행을 하나의 Document 객체로 변환할 수 있습니다.

이렇게 변환된 문서들은 이후 PDF나 웹 문서와 동일하게 Embedding, VectorStore, Retrieval 단계에서 함께 활용할 수 있습니다.

실습을 위해, CSV 파일을 먼저 Colab 환경(또는 Drive)에 업로드해주세요.

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("/content/titanic.csv")

data = loader.load()

print(len(data))
print(data[0])
```

```
891
page_content='PassengerId: 1
Survived: 0
Pclass: 3
Name: Braund, Mr. Owen Harris
Sex: male
Age: 22
SibSp: 1
Parch: 0
Ticket: A/5 21171
Fare: 7.25
Cabin:
Embarked: S' metadata={'source': '/content/titanic.csv', 'row': 0}
```

```
data[:3]
```

```
[Document(metadata={'source': '/content/titanic.csv', 'row': 0}, page_content='PassengerId: 1\nSurvived:
0\nPclass: 3\nName: Braund, Mr. Owen Harris\nSex: male\nAge: 22\nSibSp: 1\nParch: 0\nTicket: A/5 21171\nFare:
7.25\nCabin: \nEmbarked: S'),
 Document(metadata={'source': '/content/titanic.csv', 'row': 1}, page_content='PassengerId: 2\nSurvived:
1\nPclass: 1\nName: Cumings, Mrs. John Bradley (Florence Briggs Thayer)\nSex: female\nAge: 38\nSibSp: 1\nParch:
0\nTicket: PC 17599\nFare: 71.2833\nCabin: C85\nEmbarked: C'),
 Document(metadata={'source': '/content/titanic.csv', 'row': 2}, page_content='PassengerId: 3\nSurvived:
1\nPclass: 3\nName: Heikkinen, Miss. Laina\nSex: female\nAge: 26\nSibSp: 0\nParch: 0\nTicket: STON/O2.
3101282\nFare: 7.925\nCabin: \nEmbarked: S')]
```

웹베이스로더

웹베이스 로더는 웹페이지에 포함된 텍스트 콘텐츠를 직접 파싱하여 Document 객체로 변환하는 역할을 합니다.
이를 통해 뉴스 기사, 블로그 글, 공지사항과 같은 실시간으로 업데이트되는 웹 문서를 RAG 시스템의 지식 소스로 활용할 수 있습니다.
이번 실습에서는 실제 뉴스 기사를 예제로 사용하여, 웹페이지의 내용을 불러오고 텍스트 형태로 변환하는 과정을 살펴봅니다.

실습에 사용할 웹페이지는 다음과 같습니다.
https://it.chosun.com/news/articleView.html?idxno=2023092111831

```
from langchain_community.document_loaders import WebBaseLoader
```

```
WARNING:langchain_community.utils.user_agent:USER_AGENT environment variable not set, consider setting it to ide
```

```
loader = WebBaseLoader("https://it.chosun.com/news/articleView.html?idxno=2023092111831")
documents = loader.load()

print(documents[0].page_content)
```

```
기고

인터뷰


알림

전체

알립니다

인사

부음

새로나왔어요


컴퓨팅·AI

전체

일반

코딩

에듀테크

테크리포트



전체메뉴닫기
```

주석을 해제하고 코드를 실행하면, 해당 웹페이지에 포함된 본문 텍스트 전체를 불러와 확인할 수 있습니다.

웹페이지, PDF, CSV 등 서로 다른 형식의 문서들이 모두 텍스트 형태로 정상적으로 파싱된 것을 확인할 수 있습니다.

이제 이 텍스트를 **전처리(불필요한 요소 제거, 정제)** 한 뒤, Chunking과 Embedding 단계에 활용할 수 있습니다.

Step2 : TextSplitters 사용해보기

Text Splitter는 긴 텍스트 문서를 **의미를 유지한 작은 단위(Chunk)** 로 분할하는 역할을 합니다.
LLM은 한 번에 처리할 수 있는 토큰 수에 제한이 있기 때문에, 문서를 그대로 입력하는 대신 Splitter를 통해 분할된 여러 Chunk를 입력받아 처리하게 됩니다.

이 과정을 통해 긴 문서에서도 토큰 길이 제약을 극복하고, 필요한 부분만 효율적으로 검색할 수 있습니다.

분할된 각 Chunk는 이후 단계에서 1:1로 Embedding되어 VectorStore에 저장되며, 이 Chunk 단위가 RAG 시스템에서 검색과 응답의 기본 단위가 됩니다.

```
from langchain_text_splitters import CharacterTextSplitter
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

CharacterTextSplitter는 하나의 고정된 구분자(separator)를 기준으로 텍스트를 분할하는 방식입니다. 구현이 단순하고 직관적이지만, 문서 구조에 따라 분할된 Chunk가 토큰 제한을 초과하는 경우가 발생할 수 있습니다.

반면, RecursiveCharacterTextSplitter는 줄바꿈, 문장 구분자, 구두점 등 여러 구분자를 순차적으로 적용하며 텍스트를 재귀적으로 분할합니다.

이 방식은 토큰 제한을 안정적으로 만족시키는 데 유리하지만, 분할 과정에서 의미적으로 완전하지 않은 문장 단위로 잘릴 수 있다는 단점이 있습니다.

단순한 구조의 문서나, 문단 구성이 명확한 텍스트의 경우에는 CharacterTextSplitter로도 충분합니다.

하지만 실제 서비스 환경에서는 문서 길이와 구조가 제각각인 경우가 많기 때문에, 대부분의 RAG 시스템에서는 RecursiveCharacterTextSplitter를 기본 선택지로 사용합니다.

이는 Chunk 크기를 안정적으로 제어하면서도 검색 실패를 줄이는 데 유리하기 때문입니다.

```
with open("/content/state_of_the_union.txt") as f:
    text = f.read()
```

```
#len은 어떤 기준으로 chunk size를 잴 것인가?의 기준이 되는 함수입니다.
#chunk_overlap은 chunk의 앞뒤로 다른 chunk와 설정한 size까지 겹칠 수 있도록 설정하는 것입니다.
text_splitter = CharacterTextSplitter(separator="\n\n", chunk_size=1000, chunk_overlap=100, length_function = l
chunks = text_splitter.split_text(text)
```

Chunk의 내용을 확인해보겠습니다

```
print(chunks[0])
```

```
Madam Speaker, Madam Vice President, our First Lady and Second Gentleman. Members of Congress and the Cabinet. J

Last year COVID-19 kept us apart. This year we are finally together again.

Tonight, we meet as Democrats Republicans and Independents. But most importantly as Americans.

With a duty to one another to the American people to the Constitution.

And with an unwavering resolve that freedom will always triumph over tyranny.

Six days ago, Russia’s Vladimir Putin sought to shake the foundations of the free world thinking he could make i

He thought he could roll into Ukraine and the world would roll over. Instead he met a wall of strength he never

He met the Ukrainian people.

From President Zelenskyy to every Ukrainian, their fearlessness, their courage, their determination, inspires th
```

각 chunk의 길이를 확인해보겠습니다,

```
length = []
for chunk in chunks:
    length.append(len(chunk))

print(length)
```

```
[939, 995, 810, 962, 994, 883, 957, 952, 928, 915, 993, 702, 900, 950, 957, 958, 967, 996, 796, 866, 888, 966, 9
```

토큰 단위로 텍스트 분할해보기

LLM은 문장을 단어가 아닌 토큰(token) 단위로 처리합니다. 따라서 사람이 인식하는 단어 길이나 문자 수는 실제 모델이 처리하는 입력 길이와 정확히 일치하지 않을 수 있습니다.

이로 인해 문자 수나 단어 수를 기준으로 텍스트를 분할할 경우, 모델의 입력 토큰 제한을 초과하거나 예상보다 훨씬 짧은 문맥만 전달되는 문제가 발생할 수 있습니다.

실제 서비스 환경에서는 이러한 문제를 방지하기 위해, 토큰 단위를 기준으로 텍스트를 분할하는 방식을 사용합니다. 이제 토큰 기준으로 텍스트를 분할해보겠습니다.

```
!pip install tiktoken
```

```
Requirement already satisfied: tiktoken in /usr/local/lib/python3.12/dist-packages (0.12.0)
Requirement already satisfied: regex>=2022.1.18 in /usr/local/lib/python3.12/dist-packages (from tiktoken) (2025
Requirement already satisfied: requests>=2.26.0 in /usr/local/lib/python3.12/dist-packages (from tiktoken) (2.32
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from request
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.26.0->t
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.2
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.2
```

```
import tiktoken
tokenizer = tiktoken.get_encoding("cl100k_base")

def tiktoken_len(text):
    tokens = tokenizer.encode(
        text
    )
    return len(tokens)
```

```
tiktoken_length = []
for chunk in chunks:
    tiktoken_length.append(tiktoken_len(chunk))

print(length)
print(tiktoken_length)
```

```
[939, 995, 810, 962, 994, 883, 957, 952, 928, 915, 993, 702, 900, 950, 957, 958, 967, 996, 796, 866, 888, 966, 9
[197, 198, 163, 190, 203, 182, 195, 197, 206, 205, 218, 148, 188, 205, 216, 215, 209, 224, 176, 187, 201, 197, 2
```

글자 수와 토큰 수의 차이를 확인할 수 있습니다 !

Step3 : TextEmbedding 사용해보기

Embedding은 텍스트를 컴퓨터가 계산할 수 있는 수치 벡터(vector) 형태로 변환하는 과정입니다. 이 벡터는 문장의 표면적인 형태가 아니라, 의미적 유사성을 반영하도록 설계되어 있습니다.

변환된 벡터는 VectorStore에 저장되거나, 새로운 질의(Query) 벡터와의 유사도 계산을 통해 의미적으로 가까운 문서를 검색하는 데 사용됩니다.

이러한 변환은 대규모 말뭉치로 사전 학습된 Embedding 전용 모델을 통해 이루어지며, RAG 시스템에서 Retrieval 성능을 결정하는 핵심 요소입니다.

이번 실습에서는 Google의 Gemini 계열 임베딩 모델을 사용해 텍스트를 벡터로 변환해보겠습니다.

공식 문서는 아래 링크에서 확인할 수 있습니다. https://ai.google.dev/docs/embeddings_guide?hl=ko

```
import google.generativeai as genai
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/_import_hooks/_hook_injector.py:55: FutureWarning:

All support for the `google.generativeai` package has ended. It will no longer be receiving
updates or bug fixes. Please switch to the `google.genai` package as soon as possible.
See README for more details:

https://github.com/google-gemini/deprecated-generative-ai-python/blob/main/README.md

  loader.exec_module(module)
```

genai 라이브러리의 list_models 함수를 사용하여 사용 가능한 모델들의 목록을 가져옵니다.

```
import os
os.environ["sk-"] = YOUR_API_KEY
```

```
!pip install langchain-openai

from langchain_openai import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

vector = embedding_model.embed_query("hello world")

print(len(vector))
```

```
Requirement already satisfied: langchain-openai in /usr/local/lib/python3.12/dist-packages (1.1.11)
Requirement already satisfied: langchain-core<2.0.0,>=1.2.18 in /usr/local/lib/python3.12/dist-packages (from la
Requirement already satisfied: openai<3.0.0,>=2.26.0 in /usr/local/lib/python3.12/dist-packages (from langchain-
Requirement already satisfied: tiktoken<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from langchain
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: langsmith<1.0.0,>=0.3.45 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: packaging>=23.2.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.12/dist-packages (from langchain
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from langchain-c
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from langchai
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=2
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=
Requirement already satisfied: jiter<1,>=0.10.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=2.26.0->l
Requirement already satisfied: tqdm>4 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=2.26.0->la
Requirement already satisfied: regex>=2022.1.18 in /usr/local/lib/python3.12/dist-packages (from tiktoken<1.0.0,
Requirement already satisfied: requests>=2.26.0 in /usr/local/lib/python3.12/dist-packages (from tiktoken<1.0.0,
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.5.0->opena
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->openai
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0.0
```

```
Requirement already satisfied: orjson>=3.9.14 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmi
Requirement already satisfied: xxhash>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydanti
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from request
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.2
1536
```

두 개의 임베딩 모델을 사용할 수 있습니다

https://github.com/google/generative-ai-docs/blob/main/examples/gemini/python/langchain/Gemini_LangChain_QA_Chroma_WebLoad.ipynb

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings
embedding_model = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
```

Gemini embedding은 지역에 따라 사용이 제한됩니다.
주로 유럽권에서 제한되기 때문에, 다음 에러를 확인하신다면 Colab 파일의 서버 저장 위치를 확인 후, 다른 임베딩 모델로 변경해야합니다.

Error embedding content: 400 User location is not supported for the API use.

```
!curl ipinfo.io
```

```
{
  "ip": "35.189.171.71",
  "hostname": "71.171.189.35.bc.googleusercontent.com",
  "city": "Taipei",
  "region": "Taiwan",
  "country": "TW",
  "loc": "25.0531,121.5264",
  "org": "AS396982 Google LLC",
  "timezone": "Asia/Taipei",
  "readme": "https://ipinfo.io/missingauth"
}
```

```
# 400 User location is not supported for the API use 오류가 발생한다면, 이 블록을 대신 실행해주세요

# ! pip install -q sentence_transformers

#from langchain.embeddings import HuggingFaceEmbeddings
#embedding_model = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
```

embedding model 변수에 google의 임베딩모델 혹은 huggingface의 임베딩모델이 할당되었을 것입니다.
embed_documents 멤버 함수를 사용하여 새 문장을 변환해보겠습니다 (오픈AI 버전으로 코드 변경)

```
!pip install -U langchain-openai

from langchain_openai import OpenAIEmbeddings

embedding_model = OpenAIEmbeddings(
    model="text-embedding-3-small",
    api_key=YOUR_API_KEY
)

embeddings = embedding_model.embed_documents(
    [
        "This is red apple.",
        "This is yellow banana.",
        "This is green lime.",
    ]
)

print(len(embeddings))
print(len(embeddings[0]))
```

```
Requirement already satisfied: langchain-openai in /usr/local/lib/python3.12/dist-packages (1.1.11)
Requirement already satisfied: langchain-core<2.0.0,>=1.2.18 in /usr/local/lib/python3.12/dist-packages (from la
Requirement already satisfied: openai<3.0.0,>=2.26.0 in /usr/local/lib/python3.12/dist-packages (from langchain-
Requirement already satisfied: tiktoken<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from langchain
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: langsmith<1.0.0,>=0.3.45 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: packaging>=23.2.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.12/dist-packages (from langchain
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from langchain-c
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from
```

```
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from langchai
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=2
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=
Requirement already satisfied: jiter<1,>=0.10.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=2.26.0->l
Requirement already satisfied: tqdm>4 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=2.26.0->la
Requirement already satisfied: regex>=2022.1.18 in /usr/local/lib/python3.12/dist-packages (from tiktoken<1.0.0,
Requirement already satisfied: requests>=2.26.0 in /usr/local/lib/python3.12/dist-packages (from tiktoken<1.0.0,
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.5.0->opena
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->openai
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0.0
Requirement already satisfied: orjson>=3.9.14 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmi
Requirement already satisfied: xxhash>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydanti
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from request
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.2
3
1536
```

임베딩으로 잘 변환되었는지 확인해보겠습니다

```
print(embeddings[1])
```

```
[0.006427764892578125, -0.020843505859375, -0.0216217041015625, 0.0235443115234375, -0.0428466796875, -0.0045890
```

```
len(embeddings[1])
```

```
1536
```

새로운 쿼리를 넣어, 임베딩끼리 유사도를 계산해보겠습니다

```
import numpy as np
from numpy import dot
from numpy.linalg import norm
def cos_sim(A, B):
  return dot(A, B)/(norm(A)*norm(B))
```

```
query = ["this is red fruit"]
```

```
e_query = embedding_model.embed_documents(query)
print(cos_sim(embeddings[0], e_query[0]))
print(cos_sim(embeddings[1], e_query[0]))
print(cos_sim(embeddings[2], e_query[0]))
```

```
0.7478929665954975
0.4898791411166917
0.4083791543048118
```

빨간 사과와 빨간 과일의 유사도가 많이 높게 나왔습니다!

임베딩 모델은 사용 언어나 필요에 따라 다양하게 교체하여 사용할 수 있습니다.
해당 링크에서 여러 목록을 확인하실 수 있습니다.
https://python.langchain.com/docs/integrations/text_embedding/

Step4 : VectorStore 사용해보기

VectorStore는 텍스트를 Embedding 모델을 통해 벡터(vector)로 변환한 뒤, 이를 저장하고 관리하는 저장소입니다. 이 저장소는 단순한 데이터 보관 공간이 아니라, 벡터 간의 유사도를 빠르게 계산하고 탐색하기 위한 인덱싱 구조를 함께 포함하고 있습니다.

문서나 쿼리가 Embedding된 이후에는, VectorStore를 통해 의미적으로 유사한 벡터를 효율적으로 검색할 수 있으며, 이 과정이 RAG 시스템의 Retrieval 단계를 담당하게 됩니다.

대표적인 VectorStore로는 Chroma, FAISS 등이 있으며, 각각 로컬 환경과 대규모 서비스 환경에서 널리 사용됩니다.

이번 실습에서는 구성이 단순하고 로컬 환경에서 바로 사용할 수 있는 ChromaDB를 사용해 VectorStore를 구성해보겠습니다.

```
!pip install chromadb
```

```
Requirement already satisfied: typer>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.24.1)
Requirement already satisfied: kubernetes>=28.1.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (3
Requirement already satisfied: tenacity>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from chromadb) (9.1.
Requirement already satisfied: pyyaml>=6.0.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (6.0.3)
Requirement already satisfied: mmh3>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from chromadb) (5.2.1)
Requirement already satisfied: orjson>=3.9.12 in /usr/local/lib/python3.12/dist-packages (from chromadb) (3.11.
Requirement already satisfied: httpx>=0.27.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (0.28.1
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (13.9.4
Requirement already satisfied: jsonschema>=4.19.0 in /usr/local/lib/python3.12/dist-packages (from chromadb) (4
Requirement already satisfied: packaging>=24.0 in /usr/local/lib/python3.12/dist-packages (from build>=1.0.3->c
Requirement already satisfied: pyproject_hooks in /usr/local/lib/python3.12/dist-packages (from build>=1.0.3->c
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->ch
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27.0->chromadb)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.19.
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=4.1
Requirement already satisfied: six>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kubernet
Requirement already satisfied: websocket-client!=0.40.0,!=0.41.*,!=0.42.*,>=0.32.0 in /usr/local/lib/python3.12
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->ch
Requirement already satisfied: requests-oauthlib in /usr/local/lib/python3.12/dist-packages (from kubernetes>=2
Requirement already satisfied: urllib3!=2.6.0,>=1.24.2 in /usr/local/lib/python3.12/dist-packages (from kuberne
Requirement already satisfied: durationpy>=0.7 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.
Requirement already satisfied: flatbuffers in /usr/local/lib/python3.12/dist-packages (from onnxruntime>=1.14.1
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from onnxruntime>=1.14.1->c
Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-packages (from onnxruntime>=1.14.1->chro
Requirement already satisfied: importlib-metadata<8.8.0,>=6.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: googleapis-common-protos~=1.57 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: opentelemetry-exporter-otlp-proto-common==1.40.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: opentelemetry-proto==1.40.0 in /usr/local/lib/python3.12/dist-packages (from ope
Requirement already satisfied: opentelemetry-semantic-conventions==0.61b0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic>
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydant
Requirement already satisfied: python-dotenv>=0.21.0 in /usr/local/lib/python3.12/dist-packages (from pydantic-
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=1
Requirement already satisfied: huggingface-hub<2.0,>=0.16.4 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.9.0->chro
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.9.0
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.9
Requirement already satisfied: httptools>=0.6.3 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standa
Requirement already satisfied: uvloop>=0.15.1 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard
Requirement already satisfied: watchfiles>=0.20 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standa
Requirement already satisfied: websockets>=10.4 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standa
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hu
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hu
Requirement already satisfied: hf-xet<2.0.0,>=1.3.2 in /usr/local/lib/python3.12/dist-packages (from huggingfac
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.12/dist-packages (from importlib-metadata<8
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from reques
Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from requests-oauthl
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy->onnxr
```

```
#!pip install langchain-chroma
```

```
#from langchain_community.vectorstores import Chroma
```

```
# from langchain.vectorstores import Chroma
```

```
#!pip install --upgrade opentelemetry-api
#!pip install --upgrade opentelemetry-sdk
```

```
#from langchain_chroma import Chroma
```

제일 처음에 사용했던, PDF를 다시 사용하도록 합니다!

```
# 위에서 사용했던 코드입니다
# loader = PyPDFLoader("/content/Demian.pdf")
# pages = loader.load_and_split()

# text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=0, length_function = tiktoken_le
# docs = text_splitter.split_documents(pages)
```

```
#!pip show chromadb
```

Chroma에 임베딩 시킵니다

```
# db = Chroma.from_documents(docs, embedding_model)
```

이제 쿼리를 날려보겠습니다

```
# query = "how Demian look like?"
# docs = db.similarity_search(query)
```

```
# print(docs[0].page_content)
```

Face, features, looks like 등 데미안의 생김새를 담고 있는 페이지가 출력되었습니다
굉장히 빠른 속도로 검색했습니다!

Step5 : Retriever 사용해보기

Retriever는 사용자의 질문을 Embedding 모델을 통해 벡터로 변환한 뒤, VectorStore에 저장된 문서 벡터들과 비교하여 의미적으로 가장 유사한 문서(Chunk)를 찾아 반환하는 역할을 합니다.

즉, Retriever는 RAG 시스템에서 "어떤 정보를 LLM에게 참고 자료로 줄 것인가"를 결정하는 핵심 컴포넌트이며, 검색 결과의 품질이 곧 최종 답변의 품질로 이어집니다.

더블클릭 또는 Enter 키를 눌러 수정

```
!pip install -U langchain langchain-community langchain-openai
```

```
Requirement already satisfied: langgraph<1.2.0,>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: langchain-classic<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: SQLAlchemy<3.0.0,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from langc
Requirement already satisfied: requests<3.0.0,>=2.32.5 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: PyYAML<7.0.0,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from langchain-
Requirement already satisfied: aiohttp<4.0.0,>=3.8.3 in /usr/local/lib/python3.12/dist-packages (from langchain
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: dataclasses-json<0.7.0,>=0.6.7 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pydantic-settings<3.0.0,>=2.10.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: langsmith<1.0.0,>=0.1.125 in /usr/local/lib/python3.12/dist-packages (from lang
Requirement already satisfied: httpx-sse<1.0.0,>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: numpy>=1.26.2 in /usr/local/lib/python3.12/dist-packages (from langchain-commun
Requirement already satisfied: openai<3.0.0,>=2.26.0 in /usr/local/lib/python3.12/dist-packages (from langchain
Requirement already satisfied: tiktoken<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,>=3
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0,
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp<4.0.0
Requirement already satisfied: marshmallow<4.0.0,>=3.18.0 in /usr/local/lib/python3.12/dist-packages (from data
Requirement already satisfied: typing-inspect<1,>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from datac
Requirement already satisfied: langchain-text-splitters<2.0.0,>=1.1.1 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/dist-packages (from langc
Requirement already satisfied: packaging>=23.2.0 in /usr/local/lib/python3.12/dist-packages (from langchain-co
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: langgraph-checkpoint<5.0.0,>=2.1.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: langgraph-prebuilt<1.1.0,>=1.0.8 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: langgraph-sdk<0.4.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from la
Requirement already satisfied: xxhash>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from langgraph<1.2.0,>
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.
Requirement already satisfied: orjson>=3.9.14 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from langsm
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>
Requirement already satisfied: jiter<1,>=0.10.0 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=2.26.0->
Requirement already satisfied: tqdm>4 in /usr/local/lib/python3.12/dist-packages (from openai<3.0.0,>=2.26.0->
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic-
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydant
Requirement already satisfied: python-dotenv>=0.21.0 in /usr/local/lib/python3.12/dist-packages (from pydantic-
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from reques
```

```
Requirement already satisfied: regex>=2022.1.18 in /usr/local/lib/python3.12/dist-packages (from tiktoken<1.0.(
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0-
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0,
Requirement already satisfied: ormsgpack>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from langgraph-che
Requirement already satisfied: mypy-extensions>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from typing-i
```

```
!pip install -U langchain-classic
```

```
Requirement already satisfied: langchain-classic in /usr/local/lib/python3.12/dist-packages (1.0.3)
Requirement already satisfied: langchain-core<2.0.0,>=1.2.19 in /usr/local/lib/python3.12/dist-packages (from la
Requirement already satisfied: langchain-text-splitters<2.0.0,>=1.1.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: langsmith<1.0.0,>=0.1.17 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.12/dist-packages (from langchain
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from langchain-c
Requirement already satisfied: requests<3.0.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from langchain
Requirement already satisfied: sqlalchemy<3.0.0,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/dist-packages (from langcha
Requirement already satisfied: packaging>=23.2.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from langchai
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0
Requirement already satisfied: orjson>=3.9.14 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmi
Requirement already satisfied: xxhash>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydanti
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from request
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy<3.0.0,>=1
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->langsmit
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0.0
```

```
from langchain_classic.chains import RetrievalQA
print("RetrievalQA import 성공")
```

```
RetrievalQA import 성공
```

긴 문서 전체를 한 번에 LLM에 전달하는 대신, Retriever와 LLM을 결합한 RetrievalQA 체인을 사용하여 문서에서 질문과 관련된 부분만 검색하고, 그 결과를 바탕으로 답변을 생성합니다.

이를 통해 길이가 긴 문서에서도 토큰 제한을 넘지 않으면서, 근거 기반의 질의응답을 수행할 수 있습니다.

```
from langchain_core.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
from langchain_core.callbacks.manager import CallbackManager

llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    temperature=0.0,
    streaming=True,
    callback_manager=CallbackManager([StreamingStdOutCallbackHandler()])
)
```

```
WARNING:langchain_google_genai.chat_models:Unexpected argument 'callback_manager' provided to ChatGoogleGenerati
/usr/local/lib/python3.12/dist-packages/IPython/core/interactiveshell.py:3553: UserWarning: WARNING! callback_ma
                callback_manager was transferred to model_kwargs.
                Please confirm that callback_manager is what you intended.
  exec(code_obj, self.user_global_ns, self.user_ns)
```

체인의 종류와 검색(Retrieval) 방식, 그리고 그에 따른 주요 파라미터를 설정합니다.

이 단계에서는 Retriever가 어떤 전략으로 문서를 검색할지, 그리고 몇 개의 문서를 LLM에게 전달할지를 결정하게 됩니다. 이 선택은 최종 답변의 품질과 직접적으로 연결됩니다.

예를 들어, MMR(Maximal Marginal Relevance) 방식은 쿼리와의 유사도뿐만 아니라 문서 간의 중복을 줄이고 다양성을 확보하는 재정렬(Re-ranking) 전략입니다.

실무 환경에서는 단일 문서에 정보가 몰리는 것을 방지하고, LLM이 보다 풍부한 문맥을 참고하도록 하기 위해 MMR 방식이 자주 사용됩니다.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("/content/Demian-By-Hermann-Hesse.pdf")
pages = loader.load()
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

docs = text_splitter.split_documents(pages)
```

```
from langchain_community.vectorstores import Chroma

db = Chroma.from_documents(
    docs,
    embedding_model
)
```

```
qa = RetrievalQA.from_chain_type(llm, chain_type="stuff",
                                 retriever=db.as_retriever(
                                     search_type="mmr",
                                     search_kwargs={"k": 3, "fetch_k" : 10}),
                                 return_source_documents=True)
```

위 코드에서 짚고 넘어갈 파라미터는 다음과 같습니다

- chain_type="stuff"

검색된 문서(Chunk)를 그대로 하나의 Prompt에 모두 삽입하는 방식입니다. 구조가 단순하고 이해하기 쉬워, RAG 구조를 처음 학습하거나 프로토타입을 만들 때 적합합니다. 단점으로는 문서 수가 많아질 경우 토큰 사용량이 빠르게 증가할 수 있습니다. 실무에서는 초기 검증 단계에서는 stuff를, 문서 수가 많아지면 map_reduce나 refine 방식으로 확장합니다.

- retriever

VectorStore에서 어떤 문서를 검색할지 결정하는 검색 모듈입니다. 검색 전략과 파라미터 설정에 따라 LLM이 참고하는 정보의 범위와 품질이 달라집니다.

- search_type="mmr"

MMR(Maximal Marginal Relevance) 검색 방식을 사용합니다. 쿼리와의 유사도뿐만 아니라, 문서 간 중복을 줄여 다양한 문맥을 확보하는 Re-ranking 전략입니다. 실무 환경에서 단일 문서 편향을 줄이기 위해 자주 사용됩니다

- search_kwargs={"k": 3, "fetch_k": 10}
  - fetch_k
    VectorStore에서 우선적으로 가져올 후보 문서 개수입니다. Re-ranking 이전 단계에서 사용됩니다.
  - k
    최종적으로 LLM에게 전달할 문서(Chunk)의 개수입니다.

일반적으로 fetch_k > k 로 설정하여 후보 풀을 넉넉히 확보한 뒤, 품질 좋은 문서만 선별하는 방식을 사용합니다.

- return_source_documents=True

답변 생성에 사용된 원문 문서(Chunk)를 함께 반환합니다. 이를 통해 답변의 출처를 사용자에게 표시하거나 검색 품질을 디버깅하고 RAG 성능을 평가할 수 있습니다. 실무 서비스에서는 거의 필수적으로 사용하는 옵션입니다.

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,
    api_key=YOUR_API_KEY
)
```

```
from langchain_classic.chains import RetrievalQA

qa = RetrievalQA.from_chain_type(
    llm,
    chain_type="stuff",
    retriever=db.as_retriever(
        search_type="mmr",
        search_kwargs={"k": 3, "fetch_k": 10}
    ),
    return_source_documents=True
)
```

```
query = "How does Demian look like?"

result = qa.invoke({"query": query})

print(result["result"])
```

```
Demian has a wild and somewhat ugly appearance, with an interrogative and erratic demeanor. His eyes and brow co
```

마크다운 형식으로 출력해봅니다

```
from IPython.display import Markdown, display

answer = result.get("result") or result.get("answer")

display(Markdown(answer))
```

Demian has a wild and somewhat ugly appearance, with an interrogative and erratic demeanor. His eyes and brow convey masculinity and strength, while the lower half of his face is described as tender, immature, and somewhat feminine, with an irresolute, boylike chin. His dark brown eyes are full of pride and

RAG를 사용하지 않은 llm 호출도 시도해보세요!

```
from langchain_openai import ChatOpenAI
from IPython.display import Markdown, display

llm2 = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.0,
    api_key=YOUR_API_KEY
)

request = llm2.invoke("How does Demian look like?")

display(Markdown(request.content))
```

"Demian" is a novel by Hermann Hesse, published in 1919. The story follows a young man named Emil Sinclair as he navigates his journey of self-discovery and explores themes of duality, individuality, and the search for meaning.

In terms of physical appearance, the character Demian, who plays a significant role in Sinclair's life, is often described as having a striking and enigmatic presence. He is portrayed as confident, charismatic, and somewhat mysterious, embodying a sense of strength and depth that captivates Sinclair. However, specific details about his physical features are not extensively outlined in the text, allowing readers to interpret his appearance in various ways based on their own imagination.

Quiz

결과의 어떤 부분을 관찰하였을 때, RAG 시스템의 결과를 신뢰할 수 있겠다 생각하셨나요?

Answer

원문에서 답변의 출처를 확인할 수 있었습니다.

6. 완성 예제

다음은 Lagnchain으로 구현된 Question-Answer RAG 완성 예제입니다

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", forc
```

```
YOUR_API_KEY = 'sk-'
```

필요한 라이브러리를 모두 다운받습니다

```
!pip install -q langchain langchain-google-genai chromadb pypdf sentence_transformers tiktoken
```

```
!pip install U -q langchain-community langchain-core
```

```
import os
os.environ['GOOGLE_API_KEY'] = YOUR_API_KEY
```

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_classic.chains import RetrievalQA
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import Chroma
```

Text splitter 사용을 위한 준비입니다

```
import tiktoken

tokenizer = tiktoken.get_encoding("cl100k_base")

def tiktoken_len(text):
    tokens = tokenizer.encode(text)

    return len(tokens)
```

Step 1 Document loader

```
loader = PyPDFLoader("/content/Demian-By-Hermann-Hesse.pdf")
pages = loader.load_and_split()
```

Step 2 Text splitters

```
text_splitter = RecursiveCharacterTextSplitter(chunk_size=5000, chunk_overlap=50, length_function = tiktoken_le
texts = text_splitter.split_documents(pages)
```

Step 3 Vector Empeddings

```
from langchain_community.embeddings import HuggingFaceEmbeddings

embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

```
/tmp/ipykernel_9073/384917042.py:3: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was deprecate
```

```
from langchain_community.vectorstores import Chroma

docsearch = Chroma.from_documents(
    texts,
    embedding_model,
    collection_name="demian_hf_new"
)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate lim
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set
```

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 5.87kB/s]

Step 4 Retrievers

```
from langchain_core.callbacks.streaming_stdout import StreamingStdOutCallbackHandler
from langchain_google_genai import ChatGoogleGenerativeAI

# QA
llm_gemini = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    temperature=0.0
)
```

```
qa = RetrievalQA.from_chain_type(llm_gemini, chain_type="stuff",
                                 retriever=docsearch.as_retriever(
                                     search_type="mmr",
                                     search_kwargs={"k": 3, "fetch_k" : 10 }
                                 ))
```

vocab.txt: 232k/? [00:00<00:00, 13.4MB/s]

tokenizer.json: 466k/? [00:00<00:00, 25.6MB/s]

Question Answering

```
query = "how demian looks like"
result = qa(query)
```

```
from IPython.display import Markdown, display
display(Markdown(result["result"]))
```

Demian has a face that is described as ugly and somewhat wild-looking, with an interrogative and erratic appearance. His eyes and brow convey masculinity and strength, while the lower half of his face is tender, immature, and somewhat feminine. His chin is described as irresolute and boylike, which contrasts with his forehead and expression. He has dark brown eyes that are full of pride and honesty.